FACULTAD DE INGENIERÍA

UNIVERSIDAD DA VINCI DE GUATEMALA

ESTRUCTURA DE DATOS I
NOMBRE: DYLAN ODAIR PAZ NAVAS
Carné: 202503824

EXAMEN PARCIAL – ESTRUCTURA DE DATOS

Análisis Experimental de Complejidad Algorítmica (Big-O)

Introducción

En el presente trabajo se implementaron ocho algoritmos en lenguaje Java, desarrollados en versiones iterativa y recursiva: Factorial, Fibonacci, Búsqueda Lineal y Ordenamiento Burbuja.
El objetivo principal fue medir empíricamente el tiempo de ejecución de cada algoritmo para distintos tamaños de entrada, analizar su comportamiento mediante gráficas generadas en Excel e identificar su complejidad algorítmica (Big-O) a partir de los resultados experimentales.
Este análisis nos permite comparar el comportamiento práctico de los algoritmos con su fundamentación teórica.

1. Metodología

Para la medición del rendimiento se utilizó la función:

JAVA: System.nanoTime()

Se siguieron los siguientes criterios:

- Se realizaron 5 ejecuciones por cada tamaño de entrada.
- Se calculó el promedio de los tiempos obtenidos.
- No se incluyó la inicialización de datos dentro del tiempo medido.
- Se utilizaron distintos tamaños de entrada para observar el crecimiento del tiempo.
- En el caso de Burbuja se probaron escenarios: aleatorio, ordenado e inverso.
- Los resultados fueron exportados a un archivo CSV y luego organizados en Excel.
- Se generaron 4 hojas con sus respectivas gráficas y líneas de tendencia.

2. Resultados

2.1 Factorial

Las gráficas muestran un crecimiento proporcional al tamaño de entrada tanto en la versión iterativa como en la recursiva.
Complejidad observada:

- Iterativo: **O(n)**
- Recursivo: **O(n)**

El tiempo aumenta linealmente debido a que se realiza una operación por cada valor hasta llegar al caso base.

2.2 Fibonacci

En la versión iterativa el crecimiento es moderado y lineal. Y también, la versión recursiva sin memoización muestra un crecimiento acelerado conforme aumenta n.
Complejidad observada:

- Iterativo: **O(n)**
- Recursivo: **O(2^n)**

La versión recursiva genera múltiples llamadas repetidas, lo cual provoca un crecimiento exponencial visible en la gráfica.

2.3 Búsqueda Lineal

Ambas versiones recorren el arreglo elemento por elemento hasta encontrar el valor buscado o llegar al final.
Complejidad observada:

- Iterativo: **O(n)**
- Recursivo: **O(n)**

En el peor caso se revisan todos los elementos del arreglo, lo que produce crecimiento lineal.

2.4 Ordenamiento Burbuja

El algoritmo Burbuja mostró un crecimiento notablemente más pronunciado conforme aumenta el tamaño del arreglo, especialmente en escenarios aleatorio e inverso.
Complejidad observada:

- Iterativo: **$O(n^2)$**
- Recursivo: **$O(n^2)$**

El comportamiento cuadrático se debe a las comparaciones anidadas entre elementos, lo cual genera aproximadamente n(n-1)/2 operaciones.

3. Análisis Comparativo Iterativo vs Recursivo

En los algoritmos Factorial y Búsqueda Lineal, ambas versiones presentan la misma complejidad teórica. Sin embargo, la versión recursiva puede verse limitada por la profundidad de la pila de ejecución.

En Fibonacci, la diferencia es significativa: la versión recursiva sin optimización presenta un crecimiento exponencial que la hace considerablemente menos eficiente.

En Burbuja, ambas versiones mantienen el mismo orden de complejidad, ya que la estructura recursiva no reduce el número de comparaciones realizadas.

4. Conclusiones/Análisis Propio

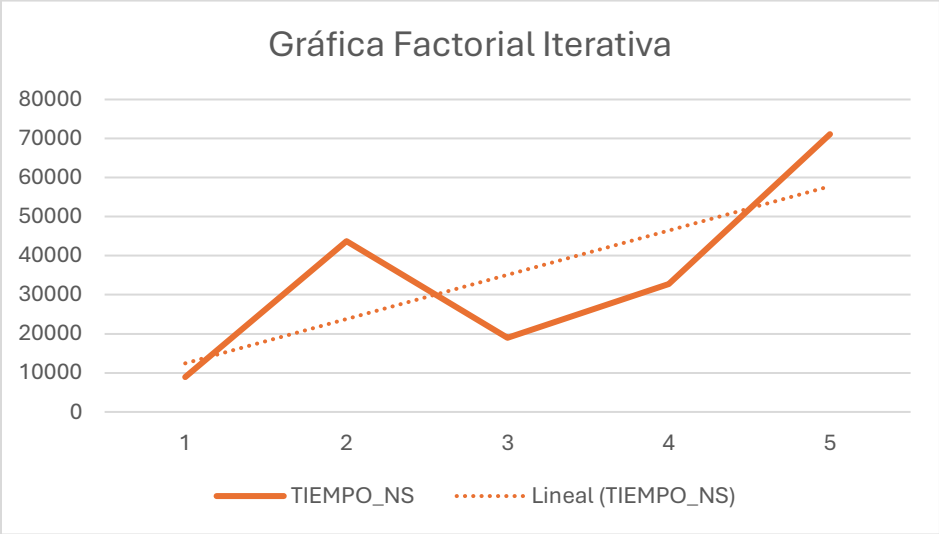
Los resultados experimentales obtenidos mediante las gráficas en Excel confirman el análisis teórico de complejidad Big-O:

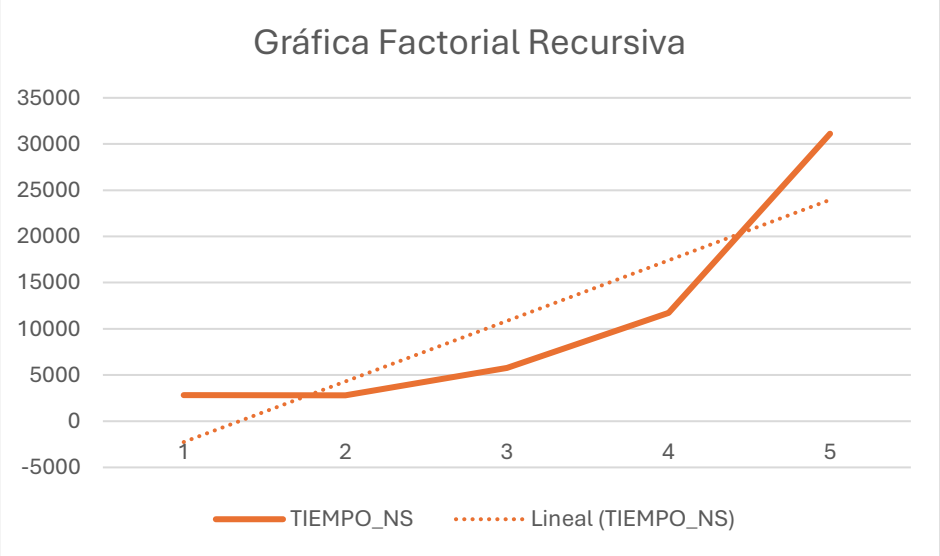
- Factorial → O(n)
- Fibonacci iterativo → O(n)
- Fibonacci recursivo → $O(2^n)$
- Búsqueda lineal → O(n)
- Burbuja → $O(n^2)$

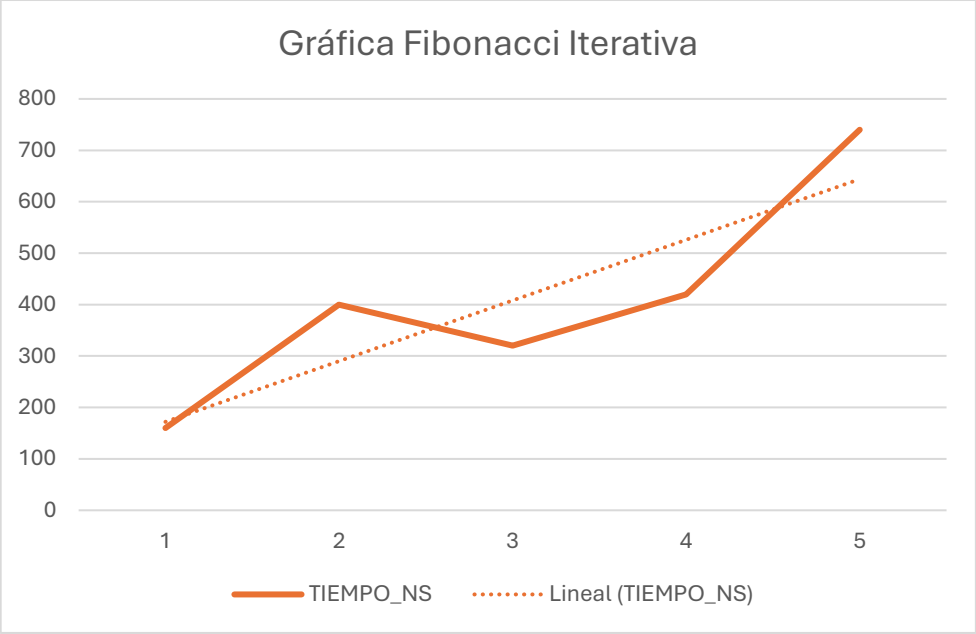
Se evidencia que el análisis experimental coincide con la teoría de complejidad algorítmica. Asimismo, se observa que las implementaciones recursivas pueden presentar limitaciones prácticas debido al uso de la pila de ejecución, aun cuando su orden de complejidad sea equivalente a la versión iterativa.

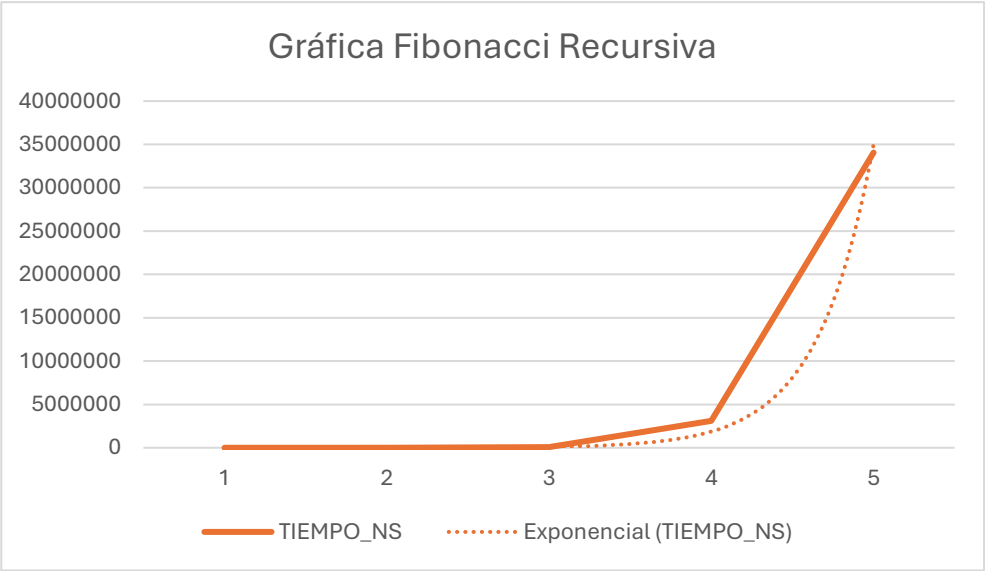
El estudio permitió comprender de manera práctica cómo el crecimiento del tiempo de ejecución refleja directamente la eficiencia del algoritmo.

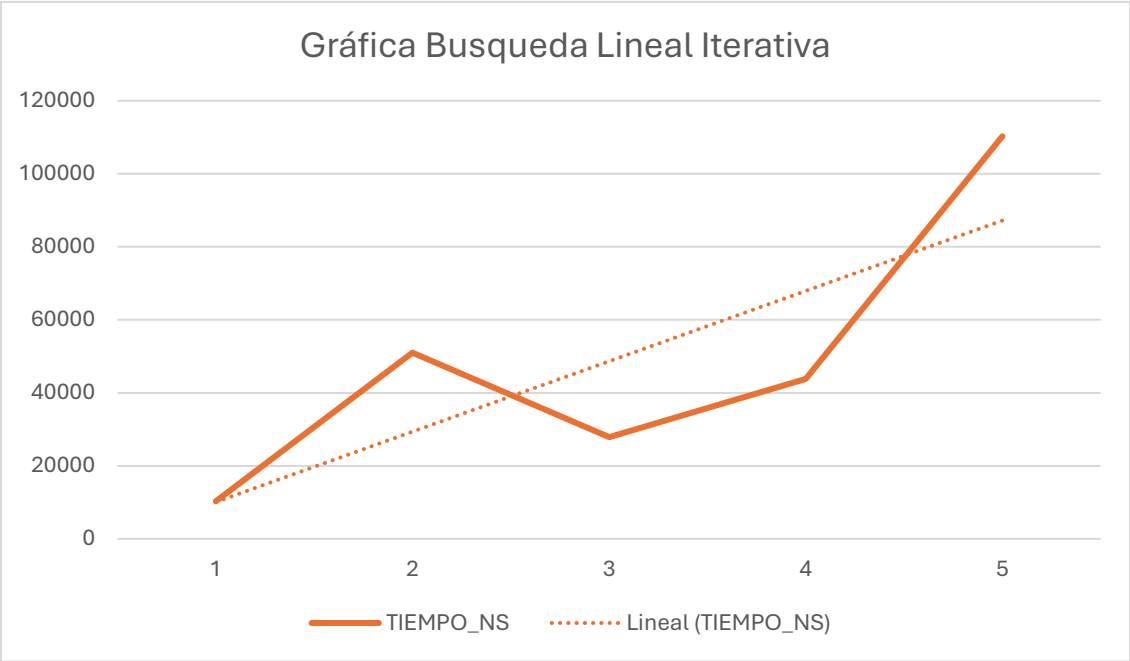
5. Gráficas

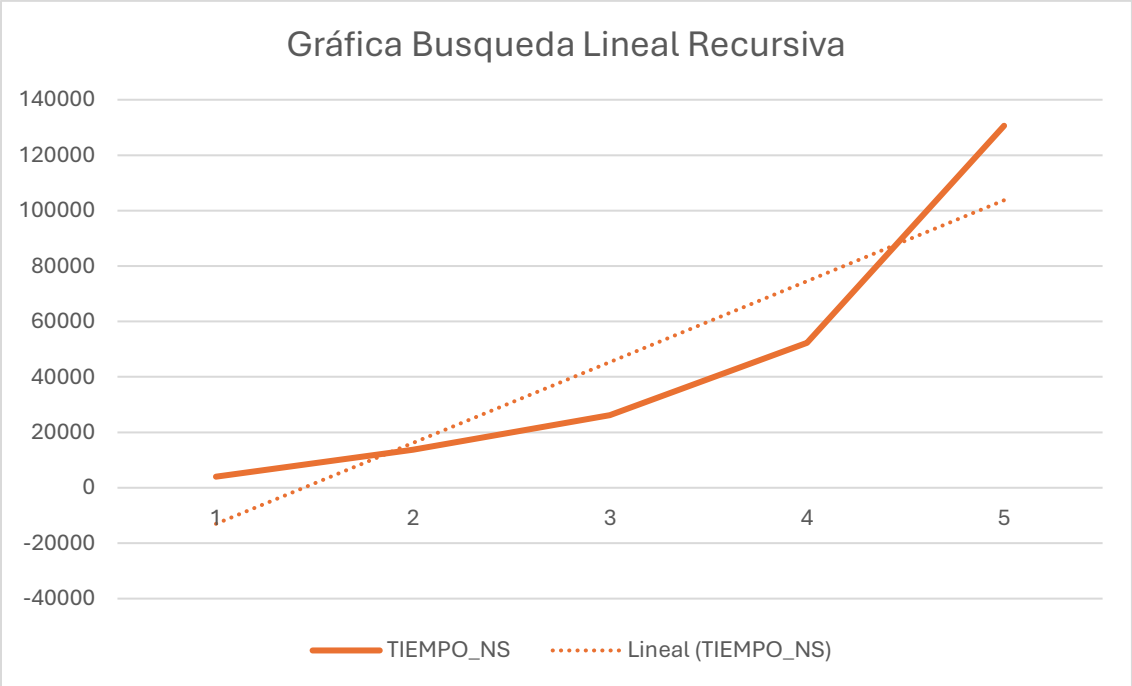


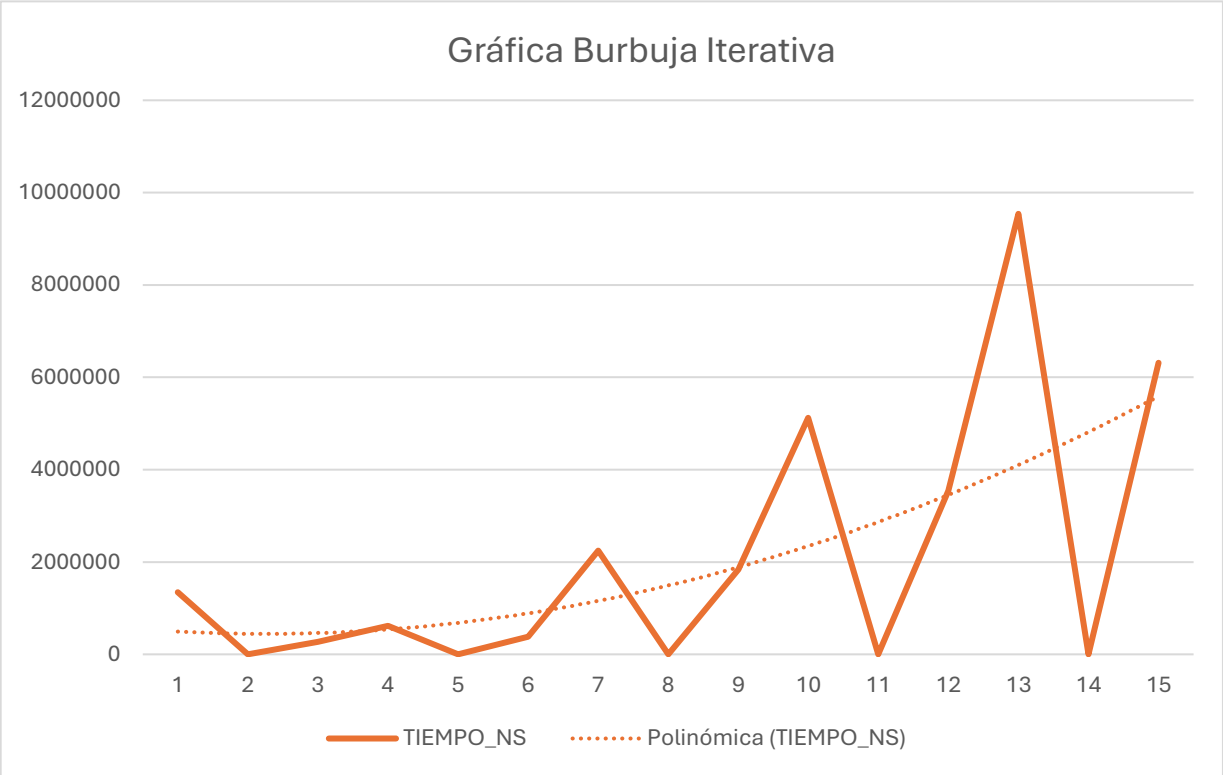


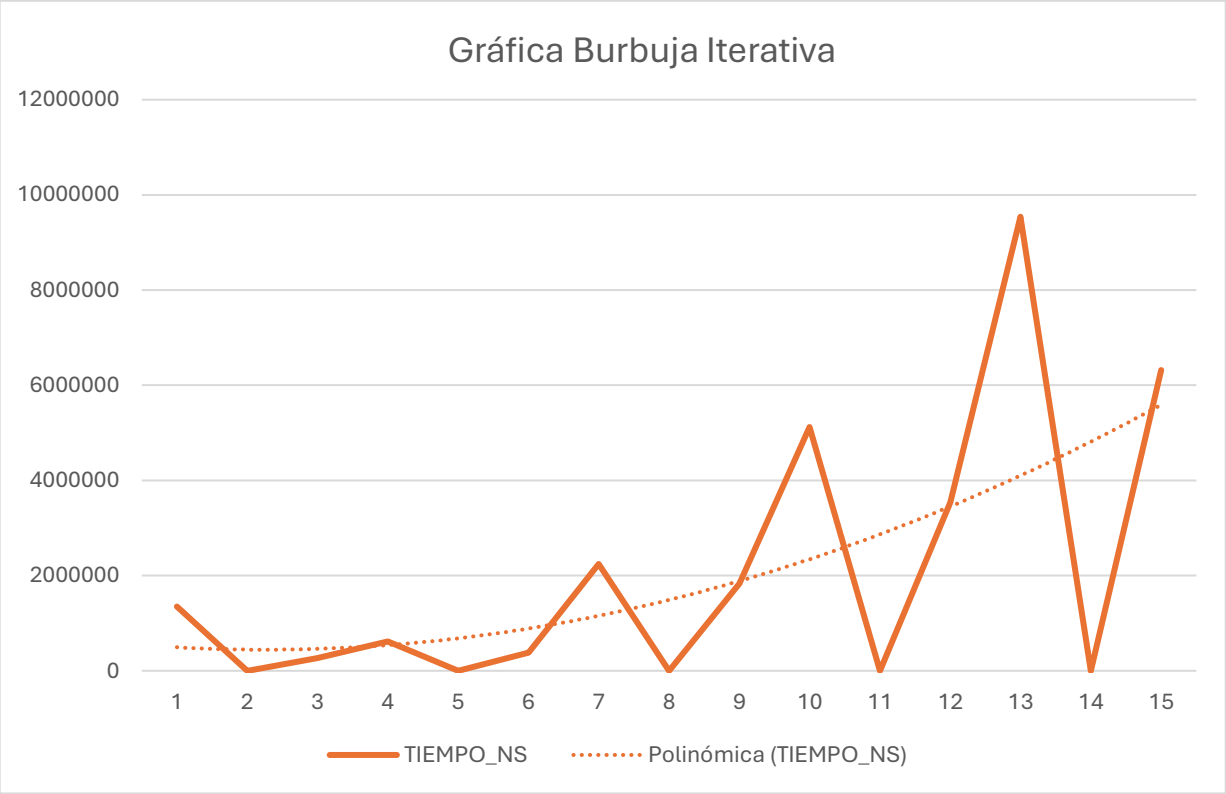

Gráfica Fibonacci Iterativa
800
700
600
500
400
300
200
100
0
1
2
3
4
5
TIEMPO_NS
Lineal (TIEMPO_NS)


Gráfica Fibonacci Recursiva
40000000
35000000
30000000
25000000
20000000
15000000
10000000
5000000
0
1
2
3
4
5
TIEMPO_NS
Exponencial (TIEMPO_NS)


Gráfica Busqueda Lineal Iterativa
120000
100000
80000
60000
40000
20000
0
1
2
3
4
5
TIEMPO_NS
Lineal (TIEMPO_NS)


Gráfica Busqueda Lineal Recursiva
140000
120000
100000
80000
60000
40000
20000
0
-20000
-40000
1
2
3
4
5
TIEMPO_NS
Lineal (TIEMPO_NS)


Gráfica Burbuja Iterativa
12000000
10000000
8000000
6000000
4000000
2000000
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
TIEMPO_NS
Polinómica (TIEMPO_NS)


Gráfica Burbuja Iterativa
12000000
10000000
8000000
6000000
4000000
2000000
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
TIEMPO_NS
Polinómica (TIEMPO_NS)